



รายงานโครงงานคอมพิวเตอร์

การเปรียบเทียบการสร้างชุดทดสอบอัตโนมัติและชุดทดสอบจาก

Generative AI บน Defects4J

An Empirical Comparison of Automated and Generative AI Test

Generation on Defects4J

โดย

673380388-2 กฤษฎา นามมนต์เพียน

673380392-1 กานดิทัต นามสุดตา

673380402-4 ดรรณภพ สุริเตอร์

อาจารย์ประจำวิชา: ผศ.ดร.ชิตสุธา สุ่มเล็ก

รายงานนี้เป็นส่วนหนึ่งของการศึกษาวิชา CP353201 Software Quality Assurance

ภาคเรียน 1 ปีการศึกษา 2569

สาขาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์

มหาวิทยาลัยขอนแก่น

บทคัดย่อ

โครงการนี้เปรียบเทียบประสิทธิภาพของการสร้างกรณีทดสอบอัตโนมัติสำหรับโปรแกรมภาษา Java ระหว่างอัลกอริทึม Automatic Test Case Generation และ Generative AI โดยใช้ Defects4J เวอร์ชัน 3.0.1 เป็น Benchmark เทคนิคที่เปรียบเทียบมี 4 เทคนิค ได้แก่ EvoSuite ซึ่งเป็น Search-Based Software Testing (ใช้อัลกอริทึม DynaMOSA), jqwik ซึ่งเป็น Property-Based Testing ที่ผู้จัดทำพัฒนาตัวสร้าง property อัตโนมัติขึ้นเอง, Gemini และ DeepSeek ซึ่งเป็น Large Language Model ที่สร้าง JUnit 4 Test Suite จาก Prompt

การเปรียบเทียบทำบนชุดข้อบกพร่องรวม 854 bugs จาก 17 projects โดยนับว่าตรวจพบข้อบกพร่องเมื่อทดสอบบน Fixed Version และไม่ผ่านบน Buggy Version อย่างไรก็ตาม EvoSuite ตัดสินระดับ Test Suite ส่วน jqwik และ LLM ตัดสินระดับเทสรายตัว และ jqwik ใช้หลาย seeds ดังรายละเอียดในบทที่ 3 ส่วน Coverage วัดด้วย JaCoCo เฉพาะคลาสที่ถูกแก้ไขในแต่ละ bug

ผลการทดลองพบว่า EvoSuite ตรวจพบข้อบกพร่องได้มากที่สุด คือ 265 bugs (31.03%) ที่ Search Budget 60 วินาที และ 275 bugs (32.2%) ที่ 120 วินาที รองลงมาคือ jqwik ซึ่งตรวจพบ 216 bugs (25.29%) ที่ 200 tries และ 230 bugs (26.93%) ที่ 1,000 tries ส่วน DeepSeek ตรวจพบ 34 bugs (3.98%) และ Gemini ตรวจพบ 31 bugs (3.63%)

สาเหตุหลักที่ Generative AI ตรวจพบข้อบกพร่องได้น้อยคือ Test Code ส่วนใหญ่คอมไพล์ไม่ผ่าน โดย Gemini คอมไพล์ผ่าน 141 จาก 854 bugs (16.51%) และ DeepSeek คอมไพล์ผ่าน 170 จาก 854 bugs (19.91%) ซึ่งอาจเกี่ยวข้องกับการที่ Prompt ไม่ได้แนบ Source Code ของคลาสเป้าหมาย เมื่อพิจารณาเฉพาะ bugs ที่คอมไพล์ผ่าน Gemini ตรวจพบข้อบกพร่อง 21.99% และ DeepSeek 20.00% อย่างไรก็ตาม ตัวหารเป็นเฉพาะ bugs ที่คอมไพล์ผ่าน จึงไม่ควรเทียบโดยตรงกับอัตราของ jqwik ที่ใช้ bugs ทั้งชุด

ด้าน Coverage EvoSuite มี Mean Line Coverage สูงที่สุด 69.88% รองลงมาคือ Gemini 59.71% (เฉพาะกรณีที่ถูกวัดได้ 137 bugs), jqwik 48.36% และ DeepSeek 46.3% ค่าเฉลี่ยเหล่านี้ใช้จำนวน bugs ที่วัด Coverage ได้ต่างกัน จึงต้องพิจารณาขอบเขตตัวอย่างร่วมด้วย นอกจากนี้ EvoSuite และ jqwik ตรวจพบข้อบกพร่องคนละกลุ่มกันในสัดส่วนที่สูง โดยมี 128 bugs ที่ตรวจพบทั้งคู่ 147 bugs ที่พบเฉพาะ EvoSuite และ 102 bugs ที่พบเฉพาะ jqwik เมื่อรวมทั้ง 4 เทคนิคตรวจพบได้ 397 จาก 854 bugs (46.49%)

โดยสรุป ไม่มีเทคนิคใดที่ดีที่สุดในทุกมิติ EvoSuite เด่นด้าน Fault Detection และ Coverage, jqwik ตรวจพบข้อบกพร่องที่ EvoSuite ไม่พบได้จำนวนมาก ส่วน Generative AI ภายใต้ Configuration ของการทดลองนี้ยังมีข้อจำกัดด้านความสามารถในการคอมไพล์ของ Test Code ที่สร้างขึ้น

คำสำคัญ — Software Testing, Automatic Test Case Generation, EvoSuite, DynaMOSA, Property-Based Testing, jqwik, Generative AI, Gemini, DeepSeek, Defects4J, Test Coverage, Fault Detection

คำนำ

รายงานโครงงานเรื่อง “AI-Assisted Testing vs. Automatic Test Case Generation Algorithms: A Benchmark and Test Coverage Evaluation” จัดทำขึ้นเป็นส่วนหนึ่งของรายวิชา CP353201 Software Quality Assurance มีวัตถุประสงค์เพื่อศึกษาและเปรียบเทียบประสิทธิภาพของเครื่องมือสร้างกรณีทดสอบอัตโนมัติกับ Generative AI ภายใต้ชุดข้อมูลและกระบวนการประเมินเดียวกัน

การศึกษานี้เปรียบเทียบ 4 เทคนิค ได้แก่ EvoSuite, jqwik, Gemini และ DeepSeek โดยให้ความสำคัญกับ Fault Detection, Line Coverage, Branch Coverage, ความสำเร็จในการคอมไพล์และรัน Test Case และปริมาณ Token ที่ Generative AI ใช้

โครงงานใช้ Defects4J เป็น Benchmark ครอบคลุม 17 projects และจัดทำ Docker Environment, Scripts, Prompt, Generated Test Code และผลการทดลองไว้ใน Repository เพื่อให้ตรวจสอบและทำซ้ำการทดลองได้

ผู้จัดทำหวังว่ารายงานฉบับนี้จะช่วยให้ผู้อ่านเข้าใจหลักการของ Search-Based Test Generation, Property-Based Testing และการใช้ Generative AI ในงาน Software Testing รวมถึงเห็นความแตกต่างระหว่าง Coverage กับ Fault Detection จากผลการทดลองจริง

ท้ายที่สุดนี้ ผู้จัดทำขอขอบพระคุณ ผู้ช่วยศาสตราจารย์ ดร.ชิตสุธา สุ่มเล็ก อาจารย์ประจำรายวิชา ที่ได้ให้คำแนะนำและข้อเสนอแนะตลอดการดำเนินโครงงาน

ผู้จัดทำ

กฤษฎา นามมนต์เทียน

กานติทัต นามสุดตา

ดริณภพ สุริเตอร์

สารบัญ

บทคัดย่อ	ก
คำนำ	ค
สารบัญ	ง
สารบัญตาราง	ช
สารบัญภาพ	ซ
บทที่ 1 บทนำ	1
1.1 ที่มาและความสำคัญ	1
1.2 วัตถุประสงค์	1
1.3 ขอบเขตของการศึกษา	2
1.4 ประโยชน์ที่คาดว่าจะได้รับ	2
1.5 นิยามศัพท์ที่เกี่ยวข้อง	3
บทที่ 2 ทฤษฎีและงานที่เกี่ยวข้อง	4
2.1 การทดสอบซอฟต์แวร์	4
2.2 การสร้างกรณีทดสอบโดยอัตโนมัติ	4
2.3 Search-Based Software Testing และ EvoSuite	4
2.4 DynaMOSA	4
2.5 Property-Based Testing และ jqwik	5
2.6 Generative AI และ Large Language Models สำหรับการสร้าง Test Case	5
2.7 Gemini และ DeepSeek	5
2.8 Defects4J	5
2.9 Test Coverage	5
2.10 Fault Detection และความสัมพันธ์กับ Coverage	6
2.11 งานวิจัยที่เกี่ยวข้อง	6
บทที่ 3 วิธีดำเนินการและการออกแบบการทดลอง	7
3.1 ภาพรวมการดำเนินงาน	7
3.2 ชุดข้อมูลและ Benchmark	7
3.3 การกำหนด Target Classes	8
3.4 สภาพแวดล้อมในการทดลอง	9
3.5 การสร้าง Test Case ด้วย EvoSuite	9

3.6 การสร้าง Test Case ด้วย jqwik	9
3.7 การออกแบบ Prompt สำหรับ Gemini และ DeepSeek	10
3.8 Configuration ของ Gemini และ DeepSeek.....	11
3.9 กระบวนการ Evaluation.....	11
3.10 หลักเกณฑ์การจำแนกผล.....	11
3.11 การวัด Test Coverage	12
3.12 ตัวชี้วัดที่ใช้ในการเปรียบเทียบ.....	12
3.13 การจัดการ Failure, Timeout และการตรวจสอบ Pipeline.....	12
3.14 การออกแบบเพื่อให้สามารถทำซ้ำได้	12
บทที่ 4 ผลการทดลอง.....	13
4.1 ภาพรวมผลการทดลอง	13
4.2 ผลด้าน Fault Detection	14
4.3 ผลของ jqwik และความคงที่ระหว่าง Seed	14
4.4 ผลของ EvoSuite	14
4.5 ผลของ Generative AI	15
4.6 ผลการวัด Test Coverage.....	16
4.7 ผลการตรวจพบข้อบกพร่องแยกตาม Project	17
4.8 ผลของข้อมูล Triggering Test ใน Prompt	18
4.9 Bugs ที่ตรวจพบร่วมกันและเฉพาะแต่ละเทคนิค	19
4.10 ผลการวิเคราะห์ Token ของ Gemini และ DeepSeek.....	20
4.11 สรุปผลการทดลองเชิงเปรียบเทียบ	21
บทที่ 5 สรุปผล อภิปรายผล และข้อเสนอแนะ	23
5.1 สรุปผลการดำเนินงาน	23
5.2 อภิปรายผลการทดลอง	23
5.3 การอภิปรายผลด้าน Token Efficiency.....	24
5.4 ผลลัพธ์การเรียนรู้	24
5.5 ปัญหาที่พบในการดำเนินงาน	24
5.6 ข้อจำกัดของการทดลอง	25
5.7 Threats to Validity.....	25
5.8 ข้อเสนอแนะและแนวทางการพัฒนาต่อ.....	26
5.9 สรุปผลโครงงาน.....	26

รายการอ้างอิง	27
ภาคผนวก ก Prompt Template	28
ภาคผนวก ข Reproducibility Guide.....	30
ภาคผนวก ค โครงสร้างผลลัพธ์	31

สารบัญตาราง

ตารางที่ 3.1 จำนวน Bugs ในแต่ละ Defects4J Project	7
ตารางที่ 3.2 สภาพแวดล้อมหลักของการทดลอง.....	9
ตารางที่ 3.3 หลักเกณฑ์การจำแนกผลการทดสอบ.....	11
ตารางที่ 4.1 ผลการทดลองภาพรวมของทั้ง 4 เทคนิค	13
ตารางที่ 4.2 ผลของ jqwik แยกตามรอบการทดลอง	14
ตารางที่ 4.3 สถานะการทำงานของ EvoSuite	15
ตารางที่ 4.4 สถานะ Evaluation ของ Gemini และ DeepSeek	15
ตารางที่ 4.5 จำนวน Bugs Detected แยกตาม Project	17
ตารางที่ 4.6 Fault Detection แยกตามการมีข้อมูล Triggering Test ใน Prompt.....	19
ตารางที่ 4.7 การซ้อนทับของ Bugs ที่ตรวจพบ.....	19
ตารางที่ 4.8 การใช้ Token และ Token Efficiency ของ Gemini และ DeepSeek	21
ตารางที่ 4.9 สรุปผลเด่นของทั้ง 4 เทคนิค.....	21
ตารางที่ ค.1 โครงสร้างไฟล์และโฟลเดอร์หลักใน Repository.....	31

สารบัญภาพ

รูปที่ 3.1 Workflow ของการทดลอง	7
รูปที่ 4.1 Overall Fault Detection Rate	13
รูปที่ 4.2 จำนวน Bugs ในแต่ละชั้นของ Gemini และ DeepSeek.....	16
รูปที่ 4.3 Mean Line Coverage และ Branch Coverage.....	17
รูปที่ 4.4 จำนวน Bugs Detected แยกตาม Project.....	18
รูปที่ 4.5 Bugs ที่ตรวจพบโดย EvoSuite 120s และ jqwik R2	20
รูปที่ 4.6 Average Total Tokens per Bug	21

บทที่ 1

บทนำ

1.1 ที่มาและความสำคัญ

การทดสอบซอฟต์แวร์เป็นกระบวนการสำคัญในการประกันคุณภาพซอฟต์แวร์ ใช้ตรวจสอบพฤติกรรมของระบบและค้นหาข้อบกพร่องก่อนส่งมอบ แต่การออกแบบและเขียนกรณีทดสอบด้วยมือใช้เวลาและแรงงานสูง โดยเฉพาะระบบที่มี Source Code ขนาดใหญ่ จึงมีการพัฒนาเทคนิค Automatic Test Case Generation ขึ้นเพื่อสร้างกรณีทดสอบโดยอัตโนมัติ

แนวทางแรกคือ Search-Based Software Testing ซึ่งใช้อัลกอริทึมค้นหาเชิงวิวัฒนาการเพื่อสร้าง Test Case ที่ครอบคลุมโค้ดให้มากที่สุด เครื่องมือที่ใช้แพร่หลายคือ EvoSuite แนวทางที่สองคือ Property-Based Testing ซึ่งผู้ทดสอบระบุคุณสมบัติที่โปรแกรมต้องเป็นจริงเสมอ แล้วให้เครื่องมือสุ่มข้อมูลนำเข้าจำนวนมากมาตรวจสอบ เครื่องมือสำหรับ Java คือ jqwik และแนวทางที่สามคือการใช้ Large Language Model เช่น Gemini และ DeepSeek สร้าง Test Code จากคำสั่งภาษาธรรมชาติ

แต่ละแนวทางมีจุดแข็งและข้อจำกัดต่างกัน และค่า Code Coverage ที่สูงไม่ได้รับประกันว่าจะตรวจพบข้อบกพร่องจริงได้ โครงการนี้จึงเปรียบเทียบทั้ง 4 เทคนิคบนข้อบกพร่องจริงจาก Defects4J ด้วยหลักการเปรียบเทียบ Fixed/Buggy ร่วมกัน โดยมีระดับการตัดสินใจและจำนวน seeds ต่างกัน เพื่อให้เห็นความสามารถในการตรวจพบข้อบกพร่อง ความครอบคลุมของโค้ด และข้อจำกัดของแต่ละเทคนิค

1.2 วัตถุประสงค์

1. ศึกษาและทดลองใช้เครื่องมือ Automatic Test Case Generation ได้แก่ EvoSuite และ jqwik
2. ศึกษาและประยุกต์ใช้ Gemini และ DeepSeek สำหรับสร้าง JUnit Test Suite
3. ดำเนินการ Benchmark กับ Defects4J จำนวน 17 Java projects บนชุดข้อบกพร่องรวม 854 bugs
4. วัดและเปรียบเทียบ Fault Detection, Line Coverage และ Branch Coverage ของแต่ละเทคนิค

5. วิเคราะห์ความสำเร็จในการคอมไพล์และรัน Test Case รวมถึง Token Usage ของ Generative AI
6. ศึกษาปัญหา ข้อจำกัด และจัดทำ Scripts, Test Code, Prompt และผลการทดลองให้ทำซ้ำได้

1.3 ขอบเขตของการศึกษา

1. ใช้ Defects4J 3.0.1 เป็นชุดข้อมูลหลัก ครอบคลุม 17 projects ซึ่งมี active bugs รวม 854 bugs
2. ทุกเทคนิค (EvoSuite, jqwik, Gemini และ DeepSeek) ทดลองบน bugs ชุดเดียวกัน ครบทั้ง 854 bugs
3. ใช้ classes.modified ของ Defects4J เป็น Target Classes
4. นิยามการตรวจพบข้อบกพร่องว่า Test ที่ผ่านบน Fixed Version และไม่ผ่านบน Buggy Version
5. วัด Line Coverage และ Branch Coverage ด้วย JaCoCo บน Fixed Version เฉพาะ Target Classes
6. EvoSuite ทดลอง 2 รอบที่ Search Budget 60 และ 120 วินาที ส่วน jqwik ทดลอง 2 รอบที่ 200 และ 1,000 tries รอบละ 3 seeds
7. Gemini และ DeepSeek ไม่มีขั้นตอนซ่อมแซม Test Code ที่คอมไพล์ไม่ผ่าน โดยอัตโนมัติ การประเมินใช้ผลที่เลือกไว้สำหรับแต่ละ bug โดยบันทึกคำขอรวมมีการสร้างซ้ำบางรายการ

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1. เข้าใจหลักการของ Search-Based Test Generation, Property-Based Testing และ การใช้ LLM สร้าง Test Case
2. เห็นข้อดีและข้อจำกัดของแต่ละเทคนิคจากผลการทดลองบนข้อบกพร่องจริง
3. เห็นความแตกต่างระหว่าง Executability, Coverage และ Fault Detection
4. ได้ Pipeline และผลการทดลองที่ตรวจสอบและทำซ้ำได้

1.5 นิยามศัพท์ที่เกี่ยวข้อง

1. Automatic Test Case Generation หมายถึง กระบวนการสร้าง Test Case โดยอัตโนมัติด้วยเครื่องมือหรืออัลกอริทึม
2. Search-Based Software Testing (SBST) หมายถึง การใช้อัลกอริทึมค้นหาเพื่อสร้าง Test Case ที่บรรลุเป้าหมาย เช่น Coverage
3. Property-Based Testing (PBT) หมายถึง การทดสอบโดยระบุคุณสมบัติที่ต้องเป็นจริงกับข้อมูลนำเข้าทุกค่า แล้วสุ่มข้อมูลนำเข้ามาตรวจสอบ
4. Defects4J หมายถึง ฐานข้อมูลข้อบกพร่องจริงของโปรแกรม Java ซึ่งแต่ละ bug มี Buggy Version และ Fixed Version
5. Fault Detection หมายถึง การที่ชุดทดสอบมี Test อย่างน้อย 1 ตัวที่ผ่านบน Fixed Version และไม่ผ่านบน Buggy Version
6. Line Coverage และ Branch Coverage หมายถึง สัดส่วนของบรรทัดและทางแยกของ Target Classes ที่ถูกรันโดยชุดทดสอบ
7. False Alarm หมายถึง Test หรือ property ที่ไม่ผ่านบน Fixed Version จึงไม่น่ามานับเป็นการตรวจพบข้อบกพร่อง
8. Seed และ Tries หมายถึง ค่าเริ่มต้นของตัวสุ่มและจำนวนข้อมูลนำเข้าที่ jqwik สุ่มต่อ 1 property

บทที่ 2

ทฤษฎีและงานที่เกี่ยวข้อง

2.1 การทดสอบซอฟต์แวร์

การทดสอบซอฟต์แวร์คือการรันโปรแกรมด้วยข้อมูลนำเข้าที่กำหนดแล้วเปรียบเทียบผลกับผลที่คาดหวัง เพื่อค้นหาข้อบกพร่องและสร้างความมั่นใจในคุณภาพ การทดสอบระดับหน่วย (Unit Testing) ตรวจสอบคลาสหรือเมธอดแต่ละส่วนแยกกัน และเป็นระดับที่เครื่องมือสร้างกรณีทดสอบอัตโนมัติส่วนใหญ่มุ่งเป้า

2.2 การสร้างกรณีทดสอบโดยอัตโนมัติ

การสร้างกรณีทดสอบโดยอัตโนมัติประกอบด้วย 2 ปัญหาหลัก คือการสร้างข้อมูลนำเข้าที่ทำให้โปรแกรมทำงานในเส้นทางต่าง ๆ และการกำหนดผลที่คาดหวัง (Test Oracle) เครื่องมือส่วนใหญ่แก้ปัญหา Oracle ด้วย Regression Oracle คือบันทึกพฤติกรรมของเวอร์ชันอ้างอิงไว้เป็น assertion แล้วใช้ตรวจจับพฤติกรรมที่เปลี่ยนไปในเวอร์ชันอื่น

2.3 Search-Based Software Testing และ EvoSuite

Search-Based Software Testing มองการสร้าง Test Case เป็นปัญหาการหาค่าเหมาะที่สุด โดยใช้ Fitness Function ที่วัดจาก Coverage Objectives เช่น บรรทัดและทางแยกที่ยังไม่ถูกครอบคลุม EvoSuite (Fraser & Arcuri, 2011) เป็นเครื่องมือ SBST สำหรับ Java ที่สร้าง JUnit Test Suite พร้อม assertion โดยอัตโนมัติ

2.4 DynaMOSA

DynaMOSA (Panichella et al., 2018) เป็นอัลกอริทึม Many-Objective ที่มองเป้าหมาย Coverage แต่ละตัวเป็น objective แยกกัน และเลือกเฉพาะเป้าหมายที่เข้าถึงได้ในขณะนั้นตาม Control Dependency ของโปรแกรมมาใช้ในการค้นหา ทำให้ใช้ Search Budget ได้คุ้มค่ากว่าการรวมทุกเป้าหมายเป็นค่าเดียว ในการทดลองนี้ใช้ EvoSuite 1.2.0 ซึ่งกำหนด DynaMOSA เป็นอัลกอริทึมเริ่มต้น สคริปต์ไม่ได้รับอัลกอริทึมอื่น และ CSV บันทึกการใช้ค่าเริ่มต้นนี้เป็น default

2.5 Property-Based Testing และ jqwik

Property-Based Testing เริ่มจาก QuickCheck (Claessen & Hughes, 2000) ผู้ทดสอบเขียน property ซึ่งเป็นข้อความที่ต้องเป็นจริงกับข้อมูลนำเข้าทุกค่า แล้วเครื่องมือสุ่มข้อมูลนำเข้าจำนวนมากเพื่อหาตัวอย่างค้าน jqwik เป็นเครื่องมือ PBT สำหรับ Java ที่ทำงานบน JUnit Platform รองรับการกำหนด seed เพื่อให้ทำซ้ำได้ และมี Arbitrary สำหรับสร้างข้อมูลหลายชนิด

ข้อจำกัดของ PBT คือ property ต้องเขียนโดยมนุษย์ โครงการนี้จึงพัฒนาตัวสร้าง property อัตโนมัติขึ้น เพื่อให้ jqwik ทำงานเป็นเครื่องมืออัตโนมัติที่เปรียบเทียบกับ EvoSuite ได้

2.6 Generative AI และ Large Language Models สำหรับการสร้าง Test Case

Large Language Model เรียนรู้จาก Source Code จำนวนมากและสร้างโค้ดจากคำสั่งภาษาธรรมชาติได้ งานวิจัยหลายชิ้นนำ LLM มาสร้าง Unit Test โดยให้ Source Code ของคลาสเป้าหมายเป็นบริบท ข้อจำกัดที่พบบ่อยคือโค้ดที่สร้างอาจคอมไพล์ไม่ผ่านเพราะอ้างถึง API ที่ไม่มีอยู่จริง และ assertion อาจไม่ตรงกับพฤติกรรมจริงของโปรแกรม

2.7 Gemini และ DeepSeek

Gemini เป็นตระกูลโมเดลของ Google ส่วน DeepSeek เป็นโมเดลของบริษัท DeepSeek ทั้งสองรองรับการสร้างโค้ดผ่าน API โครงการนี้เรียกใช้ทั้งสองโมเดลผ่านบริการ API ของมหาวิทยาลัยขอนแก่น และบันทึกจำนวน Token กับเวลาที่ใช้ของทุกคำขอ

2.8 Defects4J

Defects4J (Just et al., 2014) เป็นชุดข้อบกพร่องจริงของโปรแกรม Java แบบ Open Source แต่ละ bug มี Buggy Version, Fixed Version, รายชื่อคลาสที่ถูกแก้ไข และ Triggering Test ที่เปิดเผยข้อบกพร่อง ทำให้ประเมินได้ว่าชุดทดสอบตรวจพบข้อบกพร่องจริงหรือไม่ โดยรันชุดทดสอบเดียวกันบนทั้งสองเวอร์ชัน

2.9 Test Coverage

Test Coverage วัดว่าชุดทดสอบรันโค้ดไปมากเพียงใด Line Coverage คือสัดส่วนบรรทัดที่ถูกรัน ส่วน Branch Coverage คือสัดส่วนทางแยกของเงื่อนไขที่ถูกรัน โครงการนี้วัดทั้งสองค่าด้วย JaCoCo ซึ่งเก็บข้อมูลระดับ bytecode

2.10 Fault Detection และความสัมพันธ์กับ Coverage

Fault Detection วัดว่าชุดทดสอบเปิดเผยข้อบกพร่องจริงได้หรือไม่ Coverage สูงเพียงอย่างเดียวไม่เพียงพอที่จะยืนยันการตรวจพบข้อบกพร่อง เพราะการรันผ่านบรรทัดที่มีข้อบกพร่องไม่ได้แปลว่าข้อมูลนำเข้าจะกระตุ้นให้ผลลัพธ์ผิด และ assertion อาจไม่ได้ตรวจค่าที่ผิดนั้น การประเมินจึงควรพิจารณาทั้งสองค่าร่วมกัน

2.11 งานวิจัยที่เกี่ยวข้อง

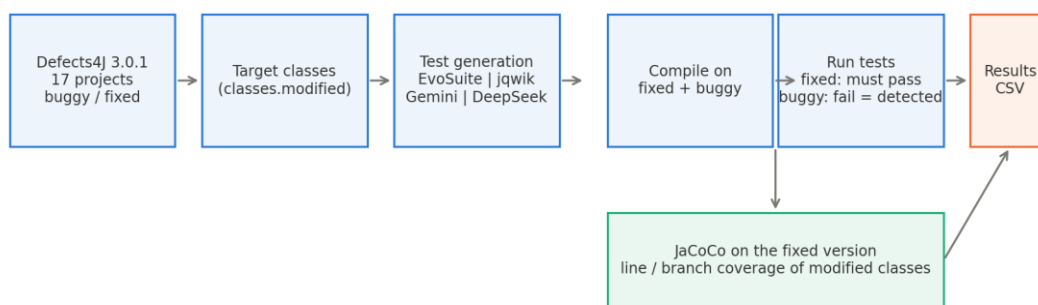
Shamshiri et al. (2015) ประเมิน EvoSuite, Randoop และ AgitarOne บน Defects4J และพบว่าเครื่องมืออัตโนมัติตรวจพบข้อบกพร่องจริงได้เพียงส่วนหนึ่ง โดยหลายกรณีรันผ่านโค้ดที่ผิดแต่ไม่มี assertion ที่จับความผิดได้ Schäfer et al. (2024) ประเมินการใช้ LLM สร้าง Unit Test และพบว่าทำให้บริบท เช่น Signature และ Source Code ของฟังก์ชัน มีผลต่อคุณภาพของ Test อย่างมาก งานเหล่านี้เป็นฐานของการออกแบบเกณฑ์ประเมินในโครงการนี้

บทที่ 3

วิธีดำเนินการและการออกแบบการทดลอง

3.1 ภาพรวมการดำเนินงาน

การทดลองแบ่งเป็น 4 ขั้นตอน คือ เตรียม Benchmark และสภาพแวดล้อม, สร้าง Test Case ด้วยแต่ละเทคนิค, ประเมิน Test Case บน Fixed Version และ Buggy Version และรวบรวมผลเป็นไฟล์ CSV เพื่อวิเคราะห์ ทุกเทคนิคใช้ Docker Image เดียวกันและหลักการเปรียบเทียบ Fixed/Buggy ร่วมกัน แต่ระดับการตัดสินใจและจำนวน seeds ต่างกันตามหัวข้อ 3.10



รูปที่ 3.1 Workflow ของการทดลอง

3.2 ชุดข้อมูลและ Benchmark

ใช้ Defects4J 3.0.1 ซึ่งมี active bugs รวม 854 bugs จาก 17 projects ทุกเทคนิคทดลองบน bugs ชุดเดียวกันครบทั้ง 854 bugs การเปรียบเทียบข้ามเทคนิคในรายงานนี้จึงใช้ bugs ทั้งหมด

ตารางที่ 3.1 จำนวน Bugs ในแต่ละ Defects4J Project

Project	Active bugs (Defects4J)	Bugs ที่ทดลอง
Chart	26	26
Cli	39	39
Closure	174	174
Codec	18	18

Project	Active bugs (Defects4J)	Bugs ที่ทดลอง
Collections	28	28
Compress	47	47
Csv	16	16
Gson	18	18
JacksonCore	26	26
JacksonDatabind	110	110
JacksonXml	6	6
Jsoup	93	93
JXPath	22	22
Lang	61	61
Math	106	106
Mockito	38	38
Time	26	26
รวม	854	854

3.3 การกำหนด Target Classes

Target Classes ของแต่ละ bug อ้างอิง classes.modified ของ Defects4J ซึ่งเป็นคลาสที่ถูกแก้ไขเพื่อซ่อมข้อบกพร่อง Coverage วัดเฉพาะคลาสในรายการนี้ ส่วนสคริปต์สร้างเทสของ LLM เลือกคลาสแรกจาก classes_modified จึงอาจสร้างเทสได้ไม่ครบทุกคลาสของ bugs ที่มีหลาย modified classes

3.4 สภาพแวดล้อมในการทดลอง

ตารางที่ 3.2 สภาพแวดล้อมหลักของการทดลอง

รายการ	ค่าที่ใช้
Container	Docker (Ubuntu 22.04)
Benchmark	Defects4J 3.0.1
Java	OpenJDK 11
Timezone	America/Los_Angeles
EvoSuite	1.2.0
jqwik	1.7.4 บน JUnit Platform 1.9.3
Coverage	JaCoCo 0.8.12
Gemini	gemini-3.5-flash-lite, gemini-3.7-flash
DeepSeek	deepseek-v4-flash
เครื่องที่ใช้รัน	Windows + Docker Desktop (WSL2), RAM 16 GB

3.5 การสร้าง Test Case ด้วย EvoSuite

EvoSuite สร้าง Test Suite จาก Fixed Version ของแต่ละ Target Class โดยใช้ criterion LINE:BRANCH:EXCEPTION อัลกอริทึมเริ่มต้น DynaMOSA ของ EvoSuite 1.2.0 (CSV บันทึก algorithm เป็น default) และ seed 1 การทดลองมี 2 รอบ คือ Search Budget 60 วินาที และ 120 วินาที สคริปต์ที่ใช้คือ scripts/run_evosuite.py และ scripts/run_benchmark.py

3.6 การสร้าง Test Case ด้วย jqwik

jqwik ไม่สร้าง property เอง ผู้จัดทำจึงพัฒนาตัวสร้าง property อัตโนมัติ (pbt.auto.Generate) ซึ่งทำงานดังนี้

1. โหลด Target Classes ของทั้ง Buggy Version และ Fixed Version ด้วย reflection แล้วสร้าง property 1 ตัวต่อ 1 เมธอดที่ไม่ใช่ private โดยใช้เฉพาะเมธอดและ constructor ที่มี signature เหมือนกันทั้งสองเวอร์ชัน
2. สุ่มข้อมูลนำเข้าด้วย jqwik เป็น “recipe” ซึ่งเป็นข้อมูลล้วน ทำให้ seed เดียวกันได้ ข้อมูลนำเข้าชุดเดียวกันทั้งสองเวอร์ชัน
3. ใช้ Regression Oracle คือบันทึกพฤติกรรมบน Fixed Version (ค่าที่ return, ชนิดของ exception, สถานะของ object และ argument ที่ถูกแก้ไข) แล้วตรวจว่า Buggy Version ให้พฤติกรรมต่างไปหรือไม่
4. รัน Fixed Version ซ้ำอีกรอบ property ที่ให้ผลไม่คงที่ เช่น ขึ้นกับค่าสุ่มหรือเวลา จะถูกตัดออกเป็น False Alarm
5. เพิ่ม contract property สำหรับ equals/hashCode และ compareTo เมื่อคลาสมีเมธอดเหล่านี้

การทดลองมี 2 รอบ คือ 200 tries และ 1,000 tries ต่อ property รอบละ 3 seeds (1, 2, 3) ตัวสร้าง property ไม่ได้รับข้อมูลเกี่ยวกับส่วนที่แก้ไขหรือ Triggering Test สคริปต์ที่ใช้คือ scripts/run_jqwik.py

3.7 การออกแบบ Prompt สำหรับ Gemini และ DeepSeek

ทั้งสองโมเดลใช้ Prompt Template เดียวกัน (ภาคผนวก ก) ซึ่งกำหนดบทบาท รูปแบบผลลัพธ์ และข้อกำหนดของ JUnit 4 ทุก Prompt มีชื่อคลาสเป้าหมาย แต่ชื่อคลาสเต็มและ package อยู่ในบล็อกบริบทซึ่งมีเพียงบางรายการ Prompt ไม่ได้แนบ Source Code ของคลาส และมีขนาดเฉลี่ยประมาณ 496 tokens ตามบันทึกของ Gemini

เมื่อจำแนก Prompt ของแต่ละโมเดลครบ 854 bugs ตามเนื้อหาที่ส่งจริง พบ 3 กลุ่ม ได้แก่ มีบล็อกบริบทพร้อมข้อมูล Triggering Failing Test และ Failure Reason จริง 55 bugs (เฉพาะ Lang), มีบล็อกบริบทแต่ข้อมูลดังกล่าวเป็น N/A 682 bugs และไม่มีบล็อกบริบท 117 bugs ตัวเลข 737 จึงหมายถึงจำนวนที่มีหัวข้อบริบท ไม่ใช่จำนวนที่มีข้อมูล Triggering Test จริง เพราะ prompt_has_hint ตรวจเพียงข้อความ Triggering Failing Test โดยไม่ตรวจค่า ผลแยกทั้ง 3 กลุ่ม รายงานในหัวข้อ 4.8

3.8 Configuration ของ Gemini และ DeepSeek

Gemini ใช้โมเดล gemini-3.5-flash-lite (783 คำขอ) และ gemini-3.7-flash (91 คำขอ) ส่วน DeepSeek ใช้ deepseek-v4-flash ทั้งหมด ทุกคำขอสร้าง Test Class 1 ไฟล์ต่อ bug สคริปต์ที่ใช้คือ scripts/run_llm_testgen.py ซึ่งบันทึก Prompt จริง, Test Code, จำนวน Token และเวลาที่ใช้

3.9 กระบวนการ Evaluation

ทุกเทคนิคขั้นตอนหลักในการประเมินคล้ายกัน คือ checkout Buggy Version และ Fixed Version ด้วย Defects4J, คอมไพล์ Test กับแต่ละเวอร์ชัน, รัน Test บน Fixed Version พร้อมเก็บ Coverage แล้วรัน Test เดิมบน Buggy Version Test ของ jqwik และ LLM รันผ่าน JUnit Platform Console Launcher ส่วน EvoSuite รันผ่าน JUnitCore

3.10 หลักเกณฑ์การจำแนกผล

ตารางที่ 3.3 หลักเกณฑ์การจำแนกผลการทดสอบ

ผลบน Fixed	ผลบน Buggy	การจำแนก
ผ่าน	ไม่ผ่าน	ตรวจพบข้อบกพร่อง (Fault Detected)
ผ่าน	ผ่าน	ไม่ตรวจพบ
ไม่ผ่าน	ไม่ผ่าน / ผ่าน	False Alarm ไม่น่ามานับ
คอมไพล์ไม่ผ่าน	-	Compile Fail นับเป็นไม่ตรวจพบ
รันไม่สำเร็จ / หมดเวลา	-	Runtime Fail / Timeout นับเป็นไม่ตรวจพบ

jqwik และ LLM ตัดสินระดับ Test รายตัว คือ bug นับว่าตรวจพบเมื่อมี Test อย่างน้อย 1 ตัวที่ผ่านบน Fixed และไม่ผ่านบน Buggy ส่วน EvoSuite ตัดสินระดับ Test Suite คือทั้งชุดต้องผ่านบน Fixed สำหรับ jqwik ซึ่งรัน 3 seeds นับว่าตรวจพบเมื่อตรวจพบอย่างน้อย 1 seed และรายงานจำนวนที่ตรวจพบในทุก seed ที่ประเมินได้สำเร็จแยกไว้ โดยไม่นับ seed ที่ประเมินไม่สำเร็จเป็นหลักฐานว่าตรวจพบหรือไม่ตรวจพบ

3.11 การวัด Test Coverage

ใช้ JaCoCo เก็บ Coverage ขณะรัน Test บน Fixed Version แล้วคำนวณ Line Coverage และ Branch Coverage เฉพาะ Target Classes ค่าเฉลี่ยที่รายงานคำนวณจาก bugs ที่วัด Coverage ได้สำเร็จเท่านั้น และรายงานจำนวน bugs ที่ใช้คำนวณกำกับไว้

3.12 ตัวชี้วัดที่ใช้ในการเปรียบเทียบ

1. Fault Detection Rate = จำนวน bugs ที่ตรวจพบ / 854
2. Mean Line Coverage และ Mean Branch Coverage ของ Target Classes
3. Compile Success Rate ของ Test Code (สำหรับ LLM)
4. จำนวน bugs ที่ตรวจพบร่วมกันและเฉพาะแต่ละเทคนิค
5. Total Tokens และจำนวน bugs ที่ตรวจพบต่อหนึ่งล้าน tokens (สำหรับ LLM)

3.13 การจัดการ Failure, Timeout และการตรวจสอบ Pipeline

การรันแต่ละครั้งมีการจำกัดเวลา กรณีที่ล้มเหลวจากสภาพแวดล้อม เช่น หน่วยความจำไม่พอ จะถูกรันซ้ำ กรณีที่ยังล้มเหลวจะบันทึกชนิดของ error และนับเป็นไม่ตรวจพบ Pipeline ของ jqwik มีชุดควบคุม 3 ชุดที่รันก่อนการทดลองจริง ได้แก่ Positive Control (Triggering Test ของ Defects4J ต้องตรวจพบเสมอ), Negative Control (property ที่ไม่เรียกโค้ดเป้าหมายต้องไม่ตรวจพบ) และ False-Alarm Control (property ที่ผิดต้องถูกตัดออก) ซึ่งผ่านทั้ง 3 ชุด

3.14 การออกแบบเพื่อให้สามารถทำซ้ำได้

Repository เก็บ Dockerfile, Scripts, Prompt, Test Code ที่สร้างขึ้น และผลการทดลอง ทุกไฟล์ jqwik กำหนด seed ทุกครั้ง การรันซ้ำด้วย seed เดิมจึงได้ข้อมูลนำเข้าชุดเดิม ขั้นตอนการทำซ้ำอยู่ในภาคผนวก ข

บทที่ 4

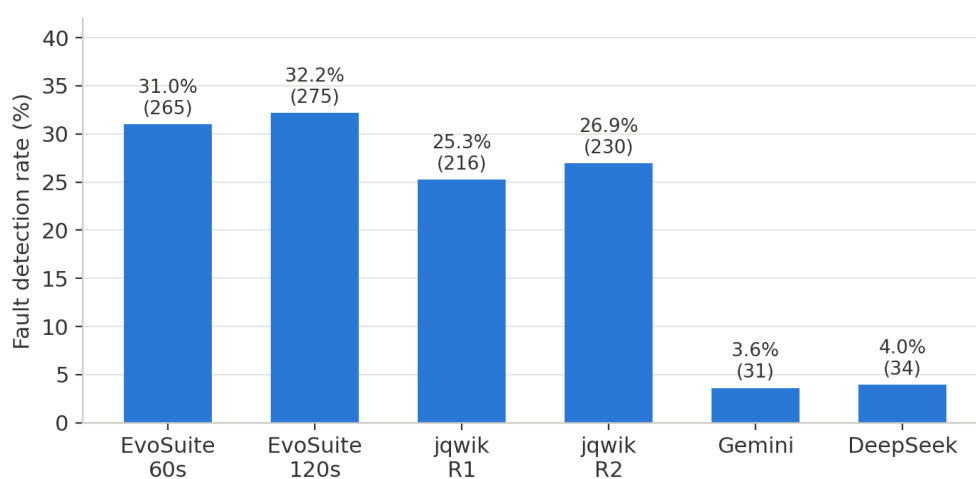
ผลการทดลอง

4.1 ภาพรวมผลการทดลอง

ตารางที่ 4.1 สรุปผลของทั้ง 4 เทคนิคบน 854 bugs EvoSuite ตรวจพบข้อบกพร่องและมีค่าเฉลี่ย Coverage สูงที่สุดในกลุ่มข้อมูลที่วัดได้ โดยจำนวน bugs ที่ใช้คำนวณ Coverage ของแต่ละเทคนิคต่างกัน jqwik ตรวจพบรองลงมา ส่วน Gemini และ DeepSeek ตรวจพบน้อยกว่ามาก

ตารางที่ 4.1 ผลการทดลองภาพรวมของทั้ง 4 เทคนิค

เทคนิค	Bugs	Detected	Detection Rate	Line Cov. (%)	Branch Cov. (%)	Bugs ที่วัด Cov.
EvoSuite 60s	854	265	31.03%	68.72	62.07	793
EvoSuite 120s	854	275	32.20%	69.88	63.83	791
jqwik R1	854	216	25.29%	46.94	37.34	830
jqwik R2	854	230	26.93%	48.36	39.17	828
Gemini	854	31	3.63%	59.71	51.56	137
DeepSeek	854	34	3.98%	46.30	39.57	163



รูปที่ 4.1 Overall Fault Detection Rate

4.2 ผลด้าน Fault Detection

EvoSuite ตรวจพบ 265 bugs ที่ 60 วินาที และ 275 bugs ที่ 120 วินาที การเพิ่ม Search Budget เป็นสองเท่าเพิ่มจำนวนที่ตรวจพบ 10 bugs jqwik ตรวจพบ 216 bugs ที่ 200 tries และ 230 bugs ที่ 1,000 tries เพิ่มขึ้น 14 bugs DeepSeek ตรวจพบ 34 bugs และ Gemini ตรวจพบ 31 bugs

4.3 ผลของ jqwik และความคงที่ระหว่าง Seed

เมื่อเพิ่มจำนวน tries จาก 200 เป็น 1,000 จำนวน bugs ที่ตรวจพบในทุก seed ที่ประเมินได้สำเร็จเพิ่มจาก 173 เป็น 211 และจำนวนที่ตรวจพบเพียงบาง seed ที่ประเมินได้สำเร็จ (Flaky) ลดจาก 43 เหลือ 19 อย่างไรก็ตาม บาง bugs ประเมินได้สำเร็จไม่ครบ 3 seeds หากนับเฉพาะ bugs ที่ตรวจพบครบทั้ง 3 seeds จะได้ 171 และ 204 bugs ตามลำดับ ผลนี้สอดคล้องกับความคงที่ที่เพิ่มขึ้นภายใต้การตั้งค่าที่ทดลอง มี 16 bugs ที่ตัวสร้าง property สร้าง property ไม่ได้

ตารางที่ 4.2 ผลของ jqwik แยกตามรอบการทดลอง

รอบ	Bugs	ตรวจพบ ≥ 1 seed	ตรวจพบทุก seed ที่ประเมินสำเร็จ	Flaky ใน valid seeds	No Property	Bugs ที่มี False Alarm
Round 1 (200 tries)	854	216	173	43	16	60
Round 2 (1,000 tries)	854	230	211	19	16	63

4.4 ผลของ EvoSuite

เมื่อเลือกผลการรันครั้งล่าสุดต่อ project, bug, Search Budget และ seed โดยไม่นับบันทึกการรันซ้ำ EvoSuite สร้าง Test ได้รวม 44,874 test cases ที่ 60 วินาที และ 47,896 test cases ที่ 120 วินาที มี 714 และ 701 bugs ตามลำดับที่ Test Suite ผ่านบน Fixed Version ครบทุกตัว ส่วนที่เหลือเป็นกรณีที่ Test Suite ไม่ผ่านบน Fixed Version หรือกระบวนการสร้างและประเมินทำงานไม่สำเร็จ จำนวน bugs ที่ตรวจพบและค่า Coverage ในตารางภาพรวมไม่เปลี่ยนเมื่อใช้ผลครั้งล่าสุดในการทดลองชุดนี้

ตารางที่ 4.3 สถานะการทำงานของ EvoSuite

Search Budget	Bugs	Test Suite ผ่านบน Fixed	Test Cases จากผลครั้งล่าสุด	Detected
60 วินาที	854	714	44,874	265
120 วินาที	854	701	47,896	275

4.5 ผลของ Generative AI

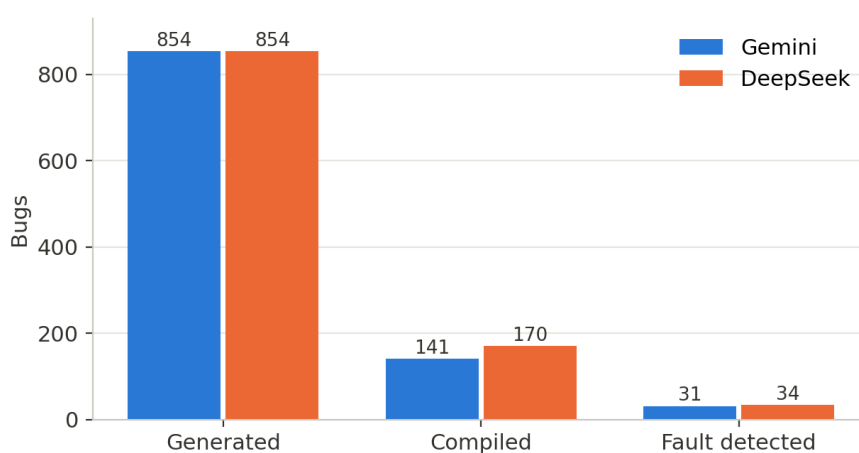
Test Code ที่ Gemini สร้างคอมไพล์ผ่าน 141 จาก 854 bugs (16.51%) และของ DeepSeek คอมไพล์ผ่าน 170 จาก 854 bugs (19.91%) error ที่พบมากที่สุดคือ cannot find symbol ซึ่งเกิดจากการเรียกเมธอดหรือคลาสที่ไม่มีใน version นั้น ซึ่งอาจเกี่ยวข้องกับการที่ Prompt ไม่ได้แนบ Source Code แต่การทดลองนี้ไม่ได้แยกวัดผลของปัจจัยดังกล่าวโดยตรง หลังคอมไพล์บน Fixed ผ่าน Gemini มีกรณีประเมินไม่สำเร็จ 4 bugs ได้แก่ หมดเวลาบน Fixed 2 bugs, Runtime Fail 1 bug และคอมไพล์ Test บน Buggy ไม่ผ่าน 1 bug ส่วน DeepSeek มี 7 bugs ได้แก่ หมดเวลาบน Fixed 4 bugs และ Runtime Fail 3 bugs

เมื่อพิจารณาเฉพาะ bugs ที่คอมไพล์ผ่าน Gemini ตรวจพบ 31 จาก 141 (21.99%) และ DeepSeek ตรวจพบ 34 จาก 170 (20.00%) Test ที่คอมไพล์ผ่านมีเทสที่ไม่ผ่านบน Fixed Version เฉลี่ย 2.5 ตัวต่อ bug สำหรับ Gemini และ 6.4 ตัวสำหรับ DeepSeek ซึ่งถูกตัดออกเป็น False Alarm

ตารางที่ 4.4 สถานะ Evaluation ของ Gemini และ DeepSeek

รายการ	Gemini	DeepSeek
Bugs ที่สร้าง Test Code	854	854
คอมไพล์ผ่าน	141 (16.51%)	170 (19.91%)
คอมไพล์ไม่ผ่าน	713	684
ประเมินไม่สำเร็จหลังคอมไพล์ Fixed ผ่าน	4	7
Test Cases ที่รันได้ (รวม)	1,892	4,387
Test Cases เฉลี่ยต่อ bug ที่รันได้	13.5	25.8

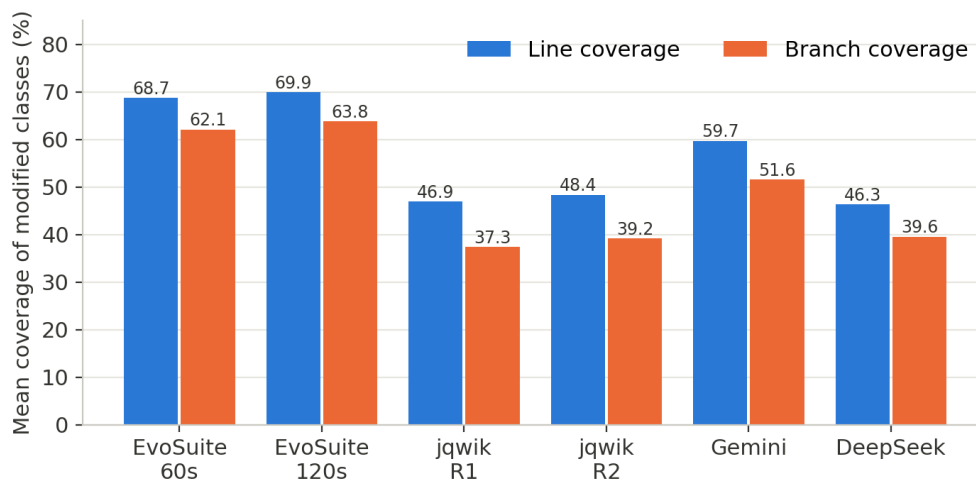
รายการ	Gemini	DeepSeek
Test ที่ไม่ผ่านบน Fixed เฉลี่ยต่อ bug ที่รันได้	2.5	6.4
Fault Detected	31	34
Detection Rate (ทั้งหมด)	3.63%	3.98%
Detection Rate (เฉพาะที่คอมไพล์ผ่าน)	21.99%	20.00%



รูปที่ 4.2 จำนวน Bugs ในแต่ละขั้นของ Gemini และ DeepSeek

4.6 ผลการวัด Test Coverage

EvoSuite มี Mean Line Coverage 68.72% และ 69.88% ที่ 60 และ 120 วินาที jqwik มี 46.94% และ 48.36% Gemini มี 59.71% และ DeepSeek มี 46.3% ค่าของ LLM คำนวณจาก bugs ที่คอมไพล์และรันสำเร็จเท่านั้น (137 และ 163 bugs) จึงเทียบกับค่าของ EvoSuite และ jqwik ซึ่งคำนวณจากมากกว่า 770 bugs โดยตรงไม่ได้



รูปที่ 4.3 Mean Line Coverage และ Branch Coverage

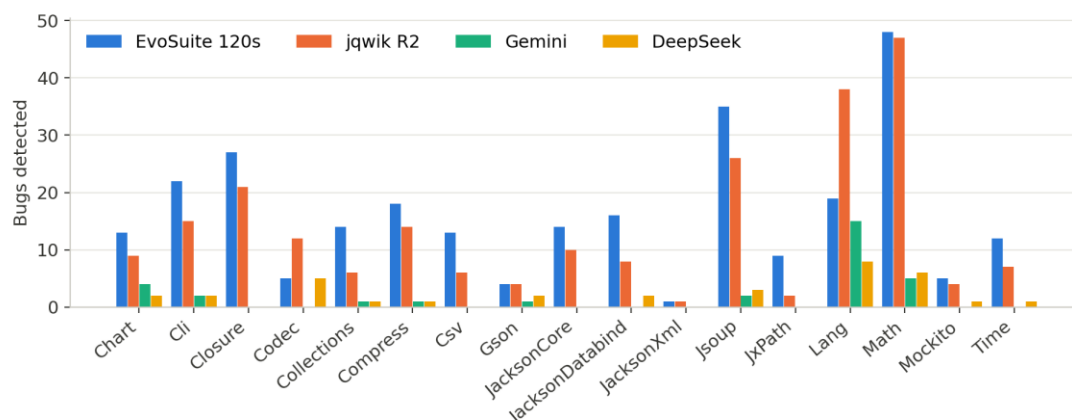
4.7 ผลการตรวจพบข้อบกพร่องแยกตาม Project

EvoSuite ตรวจพบมากกว่า jqwik ในเกือบทุก project ยกเว้น Lang และ Codec ที่ jqwik ตรวจพบมากกว่าทั้งสองรอบ และมีบาง project ที่ตรวจพบเท่ากันหรือใกล้เคียงกัน เช่น Gson, JacksonXml และ JacksonCore ที่ 60 วินาที สำหรับ Lang และ Codec คลาสส่วนใหญ่รับข้อมูลพื้นฐาน เช่น String และตัวเลข jqwik จึงทดสอบได้ลึก ส่วน project ที่ต้องใช้ object ซับซ้อน เช่น Closure, JacksonDatabind และ Mockito ทุกเทคนิคตรวจพบได้น้อย

ตารางที่ 4.5 จำนวน Bugs Detected แยกตาม Project

Project	Bugs	EvoSuite 60s	EvoSuite 120s	jqwik R1	jqwik R2	Gemini	DeepSeek
Chart	26	14	13	9	9	4	2
Cli	39	21	22	13	15	2	2
Closure	174	27	27	19	21	0	0
Codec	18	6	5	10	12	0	5
Collections	28	14	14	5	6	1	1
Compress	47	17	18	14	14	1	1
Csv	16	12	13	6	6	0	0

Project	Bugs	EvoSuite 60s	EvoSuite 120s	jqwik R1	jqwik R2	Gemini	DeepSeek
Gson	18	3	4	4	4	1	2
JacksonCore	26	10	14	10	10	0	0
JacksonDatabind	110	14	16	8	8	0	2
JacksonXml	6	1	1	1	1	0	0
Jsoup	93	37	35	25	26	2	3
JXPath	22	8	9	2	2	0	0
Lang	61	18	19	36	38	15	8
Math	106	47	48	43	47	5	6
Mockito	38	5	5	4	4	0	1
Time	26	11	12	7	7	0	1
รวม	854	265	275	216	230	31	34



รูปที่ 4.4 จำนวน Bugs Detected แยกตาม Project

4.8 ผลของข้อมูล Triggering Test ใน Prompt

กลุ่มที่มีข้อมูล Triggering Test และ Failure Reason จริงมี 55 bugs โดย Gemini ตรวจพบ 13 bugs (23.64%) และ DeepSeek 8 bugs (14.55%) กลุ่มที่มีบล็อกริบทแต่ค่าเป็น N/A มี 682 bugs ตรวจพบ 16 (2.35%) และ 21 (3.08%) ตามลำดับ ส่วนกลุ่มที่ไม่มีบล็อกริบทมี 117

bugs ตรวจพบ 2 (1.71%) และ 5 (4.27%) แม้กลุ่มแรกมีอัตราตรวจพบสูงกว่า แต่เป็นข้อมูลเฉพาะ Lang และแต่ละกลุ่มประกอบด้วย bugs ต่างกัน จึงยังสรุปไม่ได้ว่าข้อมูล Triggering Test เป็นสาเหตุของความแตกต่าง

ตารางที่ 4.6 Fault Detection แยกตามการมีข้อมูล Triggering Test ใน Prompt

โมเดล	กลุ่ม Prompt	Bugs	Detected	Rate
Gemini	Triggering Test จริง	55	13	23.64%
Gemini	บริบท + N/A	682	16	2.35%
Gemini	ไม่มีบริบท	117	2	1.71%
DeepSeek	Triggering Test จริง	55	8	14.55%
DeepSeek	บริบท + N/A	682	21	3.08%
DeepSeek	ไม่มีบริบท	117	5	4.27%

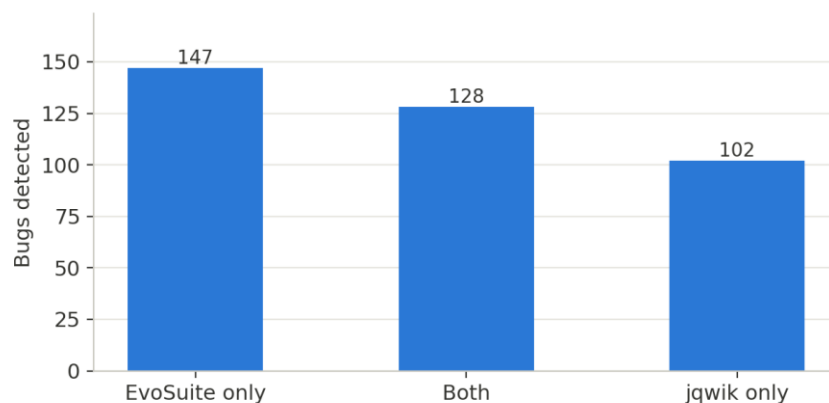
4.9 Bugs ที่ตรวจพบร่วมกันและเฉพาะแต่ละเทคนิค

เมื่อเปรียบเทียบ EvoSuite 120 วินาทีกับ jqwik Round 2 มี 128 bugs ที่ตรวจพบทั้งคู่ 147 bugs ที่พบเฉพาะ EvoSuite และ 102 bugs ที่พบเฉพาะ jqwik รวมสองเทคนิคตรวจพบ 377 bugs (44.15%) Gemini และ DeepSeek ตรวจพบร่วมกัน 62 bugs โดยมี 20 bugs ที่ EvoSuite และ jqwik ไม่พบ เมื่อรวมทั้ง 4 เทคนิคตรวจพบ 397 bugs (46.49%) และมี 457 bugs ที่ไม่มีเทคนิคใดตรวจพบ

ตารางที่ 4.7 การซ้อนทับของ Bugs ที่ตรวจพบ

กลุ่ม	จำนวน Bugs
ตรวจพบทั้ง EvoSuite 120s และ jqwik R2	128
ตรวจพบเฉพาะ EvoSuite 120s	147
ตรวจพบเฉพาะ jqwik R2	102
รวม EvoSuite และ jqwik	377
ตรวจพบโดย Gemini หรือ DeepSeek	62

กลุ่ม	จำนวน Bugs
ตรวจพบเฉพาะ LLM (EvoSuite และ jqwik ไม่พบ)	20
รวมทั้ง 4 เทคนิค	397
ไม่มีเทคนิคใดตรวจพบ	457



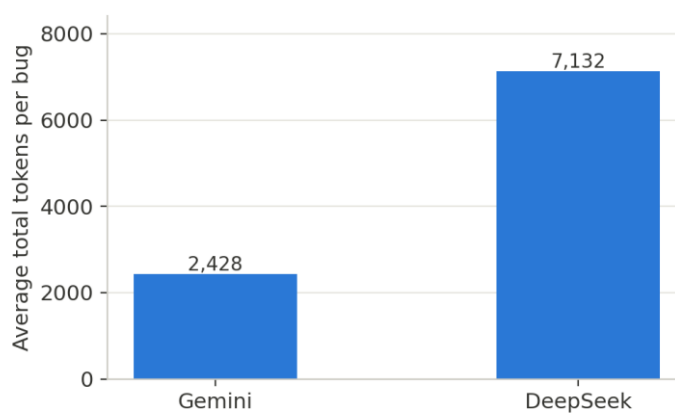
รูปที่ 4.5 Bugs ที่ตรวจพบโดย EvoSuite 120s และ jqwik R2

4.10 ผลการวิเคราะห์ Token ของ Gemini และ DeepSeek

Gemini ใช้ Total Tokens 2,073,673 tokens จาก 874 คำขอ ส่วน DeepSeek ใช้ 6,090,934 tokens จาก 863 คำขอ DeepSeek สร้าง Completion ยาวกว่า Gemini ประมาณ 3.6 เท่า และมีผลรวมระยะเวลาของคำขอมากกว่า เมื่อคิดเป็นจำนวน bugs ที่ตรวจพบต่อหนึ่งล้าน tokens Gemini ได้ 14.95 และ DeepSeek ได้ 5.58 ตัวเลข Token รวมจากบันทึกคำขอทั้งหมด ซึ่งมีจำนวนคำขอมากกว่าจำนวน bugs และอาจรวมการสร้างซ้ำ อัตรานี้จึงเป็นสถิติรวมของบันทึก ไม่ใช่ต้นทุนที่เชื่อมโยงกับแต่ละ bug ที่ตรวจพบโดยตรง Total Tokens ใช้ค่า usage.total_tokens ที่ API รายงานโดยตรง สำหรับ Gemini ผลรวม Prompt และ Completion Tokens เท่ากับ 2,014,451 tokens ซึ่งต่ำกว่า Total Tokens ที่รายงาน 59,222 tokens โดยข้อมูลที่บันทึกไม่ได้ แยกแยะส่วนต่างนี้ เวลาในตารางเป็นผลรวม duration_sec ของทุกคำขอ ไม่ใช่เวลาตั้งแต่เริ่มจนจบ การทดลองเมื่อมีการรันพร้อมกัน

ตารางที่ 4.8 การใช้ Token และ Token Efficiency ของ Gemini และ DeepSeek

รายการ	Gemini	DeepSeek
จำนวนคำขอ	874	863
Prompt Tokens	433,640	434,630
Completion Tokens	1,580,811	5,656,304
Total Tokens	2,073,673	6,090,934
Total Tokens เฉลี่ยต่อ bug	2,428	7,132
ผลรวมเวลาคำขอ (ชั่วโมง)	1.99	47.36
Fault Detected	31	34
Detected ต่อหนึ่งล้าน tokens	14.95	5.58



รูปที่ 4.6 Average Total Tokens per Bug

4.11 สรุปผลการทดลองเชิงเปรียบเทียบ

ตารางที่ 4.9 สรุปผลเด่นของทั้ง 4 เทคนิค

เทคนิค	จุดเด่น	ข้อจำกัด
EvoSuite	Fault Detection และ Coverage สูงที่สุด	มีกรณีที่ Test Suite ไม่ผ่านบน Fixed หรือกระบวนการทำงานไม่สำเร็จ

เทคนิค	จุดเด่น	ข้อจำกัด
jqwik	ตรวจพบ 102 bugs ที่ EvoSuite ไม่พบ และผลทำซ้ำได้ด้วย seed	สร้าง property สำหรับคลาสที่ต้องใช้ object ซับซ้อนได้จำกัด และมี Coverage เฉลี่ยต่ำกว่าในกลุ่มที่วัดได้
Gemini	เมื่อคอมไพล์ผ่านมี Coverage สูง และใช้ Token น้อย	คอมไพล์ผ่านเพียง 16.51%
DeepSeek	ตรวจพบมากกว่า Gemini เล็กน้อย และคอมไพล์ผ่านมากกว่า	ใช้ Token และเวลามากกว่า Gemini หลายเท่า

บทที่ 5

สรุปผล อภิปรายผล และข้อเสนอแนะ

5.1 สรุปผลการดำเนินงาน

โครงการนี้เปรียบเทียบ EvoSuite, jqwik, Gemini และ DeepSeek บน Defects4J 17 projects ด้วยหลักการเปรียบเทียบ Fixed/Buggy ร่วมกัน แต่มีระดับการตัดสินใจและจำนวน seeds ต่างกัน บน 854 bugs EvoSuite ตรวจพบข้อบกพร่องมากที่สุด (32.2%) รองลงมาคือ jqwik (26.93%) DeepSeek (3.98%) และ Gemini (3.63%) และ EvoSuite มีค่าเฉลี่ย Coverage สูงที่สุดในข้อมูลที่วัดได้ ซึ่งใช้กลุ่ม bugs ต่างกันในแต่ละเทคนิค

5.2 อภิปรายผลการทดลอง

ประการแรก EvoSuite และ jqwik ตรวจพบข้อบกพร่องคนละกลุ่มกันในสัดส่วนที่สูง jqwik ตรวจพบ 102 bugs ที่ EvoSuite 120s ไม่พบ แม้ค่าเฉลี่ย Line Coverage ที่รายงานจะต่ำกว่า ประมาณ 21.5 จุดเปอร์เซ็นต์ โดยค่าเฉลี่ยดังกล่าวใช้กลุ่ม bugs ที่วัด Coverage ได้ต่างกัน ผลที่ตรวจพบต่างกลุ่มอาจเกี่ยวข้องกับการสุ่มข้อมูลนำเข้าหลายค่าของ jqwik และการมุ่งเพิ่ม Coverage ของ EvoSuite แต่การทดลองนี้ไม่ได้แยกวัดสาเหตุโดยตรง ผลการรวมชุด bugs ที่ตรวจพบสนับสนุนว่าการใช้สองเทคนิคร่วมกันเพิ่มจำนวนข้อบกพร่องที่ตรวจพบได้ภายใต้ Configuration นี้

ประการที่สอง การเพิ่ม budget ให้ผลเพิ่มไม่มาก EvoSuite ตรวจพบเพิ่ม 10 bugs เมื่อเพิ่มเวลาเป็นสองเท่า และ jqwik ตรวจพบเพิ่ม 14 bugs เมื่อเพิ่ม tries เป็นห้าเท่า แต่สำหรับ jqwik budget ที่มากขึ้นทำให้ผลคงที่ขึ้นอย่างชัดเจน

ประการที่สาม ผลของ Generative AI ถูกจำกัดด้วยการคอมไพล์เป็นหลัก มีเพียง 16.51% และ 19.91% ของ Test Code ที่คอมไพล์ผ่าน ซึ่งอาจเกี่ยวข้องกับการที่โมเดลไม่ได้รับ Source Code และอ้างอิง API ไม่ตรงกับ version นั้น เมื่อคอมไพล์ผ่าน อัตราการตรวจพบอยู่ที่ประมาณ 20–22% แต่ใช้ตัวหารเฉพาะ bugs ที่คอมไพล์ผ่าน จึงไม่ควรเทียบโดยตรงกับอัตราของ jqwik ที่ใช้ bugs ทั้งหมด ผลนี้จึงสะท้อนข้อจำกัดของ Configuration มากกว่าความสามารถสูงสุดของโมเดล

5.3 การอภิปรายผลด้าน Token Efficiency

Gemini ตรวจพบ 14.95 bugs ต่อหนึ่งล้าน tokens ส่วน DeepSeek ตรวจพบ 5.58 DeepSeek สร้าง Test Case ต่อ bug มากกว่าประมาณสองเท่า แต่จำนวนที่ตรวจพบเพิ่มขึ้นเพียงเล็กน้อย การสร้าง Test จำนวนมากขึ้นจึงไม่ได้ให้ผลคุ้มค่าตามสัดส่วนภายใต้ Configuration นี้

5.4 ผลลัพธ์การเรียนรู้

1. การประเมินเครื่องมือสร้าง Test ต้องรันบนทั้ง Buggy และ Fixed Version จึงแยก Test ที่ผิดออกจาก Test ที่ตรวจพบข้อบกพร่องจริงได้
2. ชุดควบคุม (Positive, Negative, False-Alarm Control) ช่วยตรวจว่า Pipeline ประเมินผลถูกต้องก่อนเชื่อถือตัวเลข
3. พฤติกรรมที่ไม่คงที่ เช่น ค่าสุ่ม เวลา และ identity hash ต้องถูกคัดออกก่อนใช้ Regression Oracle
4. อัตราการคอมไพล์ผ่านที่ต่ำชี้ว่าควรทดลองเพิ่มบริบท เช่น Source Code ใน Prompt และวัดผลเปรียบเทียบโดยตรง เพื่อประเมินว่าบริบทดังกล่าวช่วยเพิ่มคุณภาพของ Test Code ได้เพียงใด

5.5 ปัญหาที่พบในการดำเนินงาน

1. ผลรอบแรกของ jqwik ใช้ไม่ได้ เพราะสคริปต์เดิมรัน Developer Tests ของ project แทน property จึงต้องเขียน Pipeline ใหม่
2. Docker Desktop จำกัดหน่วยความจำ ทำให้การรันหลายงานพร้อมกันล้นเหลว ต้องปรับหน่วยความจำและลดจำนวนงานที่รันพร้อมกัน แล้วรันซ้ำเฉพาะงานที่ล้นเหลว
3. รายการ bugs ใน dataset ช่วงแรกขาด bug หมายเลข 1 ของ 16 projects ทำให้จำนวน bugs ของแต่ละเทคนิคไม่เท่ากัน จึงต้องเพิ่มรายการและรันเพิ่มให้ทุกเทคนิค ครบ 854 bugs
4. สคริปต์สร้าง Prompt ให้ข้อมูล Triggering Test และ Failure Reason จริงเฉพาะ Lang 55 bugs อีก 682 bugs มีค่า N/A และ 117 bugs ไม่มีบล็อบริบท การใช้ prompt_has_hint เป็นตัวแทนว่ามีข้อมูลจริงจึงทำให้จำแนกผิด ต้องแยกทั้ง 3 กลุ่มตามเนื้อหา Prompt และทุกกลุ่มไม่ได้รับ Source Code

5.6 ข้อจำกัดของการทดลอง

1. EvoSuite ใช้ seed เดียว ส่วน jqwik ใช้ 3 seeds และนับว่าตรวจพบเมื่อพบอย่างน้อย 1 seed ซึ่งเป็นเกณฑ์ที่เอื้อต่อ jqwik
2. EvoSuite ตัดสินระดับ Test Suite ส่วน jqwik และ LLM ตัดสินระดับ Test รายตัว
3. Prompt ของ LLM สำหรับ Lang 55 จาก 854 bugs มีข้อมูล Triggering Test และ Failure Reason จริง ซึ่ง EvoSuite และ jqwik ไม่ได้รับ อีก 682 bugs มีบล็อกริบทแต่ค่าเป็น N/A และ 117 bugs ไม่มีบล็อกริบท ทุกกลุ่มไม่ได้รับ Source Code ผลแยกกลุ่มจึงมีทั้งความต่างของบริบทและความต่างของ project/bug
4. LLM ไม่มีขั้นตอนซ่อมแซม Test ที่คอมไพล์ไม่ผ่านโดยอัตโนมัติ และบันทึกคำขอมีการสร้างซ้ำบางรายการ Token และเวลารวมจึงครอบคลุมคำขอทั้งหมด ไม่ใช่เฉพาะ Test Code ที่ใช้ประเมินครั้งสุดท้าย
5. jqwik และ EvoSuite ใช้ Regression Oracle ที่สร้างจาก Fixed Version จึงวัดความสามารถในการกระตุ้นพฤติกรรมที่ต่างกัน ไม่ได้วัดความถูกต้องเทียบกับ Specification
6. บาง project ของ jqwik รันในช่วงที่หน่วยความจำไม่พอ ซึ่งอาจทำให้ตรวจพบน้อยกว่าที่ควรเล็กน้อย

5.7 Threats to Validity

Internal Validity: ความแตกต่างของเกณฑ์ตัดสินระหว่างเทคนิค (ระดับ Suite กับระดับ Test และจำนวน seed) อาจส่งผลต่อการเปรียบเทียบ เช่นเดียวกับการให้ข้อมูล Triggering Test จริงเฉพาะ Lang 55 bugs ผู้จัดทำรายงานจำนวนที่ jqwik ตรวจพบในทุก seed ที่ประเมินสำเร็จและครบทั้ง 3 seeds แยกกัน และจำแนก Prompt เป็นกลุ่มข้อมูลจริง, N/A และไม่มีบล็อกริบท อย่างไรก็ตาม การรายงานแยกกลุ่มยังไม่สามารถแยกผลของข้อมูล Triggering Test ออกจากผลของ project และความยากของ bug ได้

External Validity: ผลการทดลองมาจาก 17 projects ใน Defects4J ซึ่งเป็นโครงการภาษา Java จึงอาจไม่สะท้อนซอฟต์แวร์ประเภทอื่น และ LLM อาจเคยเห็น Source Code และ Test ของ projects เหล่านี้ระหว่างการฝึก

Construct Validity: Fault Detection วัดจากความต่างของพฤติกรรมระหว่างสองเวอร์ชัน ซึ่งเป็นตัวแทนของการตรวจพบข้อบกพร่องในงานจริงได้เพียงส่วนหนึ่ง เพราะในงานจริงไม่มี Fixed Version ให้ใช้เป็น Oracle

5.8 ข้อเสนอแนะและแนวทางการพัฒนาต่อ

1. แแนบ Source Code ของ Target Classes ใน Prompt และใช้โครงสร้างบริบทเดียวกัน สำหรับทุก bug โดยตัดข้อมูล Triggering Test และ Failure Reason จริงออกจาก Lang 55 bugs แล้วสร้างและประเมิน Test ใหม่ เพื่อศึกษาผลภายใต้เงื่อนไขการให้บริบทที่เทียบเคียงกัน
2. เพิ่มขั้นตอนซ่อมแซม Test ที่คอมไพล์ไม่ผ่านโดยส่ง error กลับให้ LLM
3. รัน EvoSuite หลาย seed เพื่อให้เปรียบเทียบกับ jqwik ที่ใช้ 3 seeds ได้อย่างเป็นธรรมชาติ
4. ปรับตัวสร้าง property ของ jqwik ให้สร้าง object ซับซ้อนได้มากขึ้น เช่น Reader, Stream และ reflection Type
5. ศึกษาการใช้ LLM เขียน property สำหรับ jqwik เพื่อรวมจุดแข็งของสองแนวทาง

5.9 สรุปผลโครงการ

ภายใต้ Configuration ของการทดลองนี้ EvoSuite เป็นเทคนิคที่ตรวจพบข้อบกพร่องและครอบคลุมโค้ดได้มากที่สุด jqwik ที่ใช้ property ซึ่งสร้างอัตโนมัติตรวจพบได้ใกล้เคียงและพบข้อบกพร่องที่ EvoSuite ไม่พบจำนวนมาก ส่วน Gemini และ DeepSeek ถูกจำกัดด้วยอัตราการคอมไพล์ผ่านที่ต่ำ เมื่อรวมทั้ง 4 เทคนิคตรวจพบข้อบกพร่องได้ 46.49% ของ bugs ทั้งหมด ผลการศึกษาจึงสนับสนุนการใช้หลายเทคนิคร่วมกัน และการประเมินด้วยตัวชี้วัดหลายด้านแทนการดู Coverage เพียงอย่างเดียว

รายการอ้างอิง

- Claessen, K., & Hughes, J. (2000). QuickCheck: A lightweight tool for random testing of Haskell programs. Proceedings of the 5th ACM SIGPLAN International Conference on Functional Programming (ICFP), 268–279.
- Fraser, G., & Arcuri, A. (2011). EvoSuite: Automatic test suite generation for object-oriented software. Proceedings of the 19th ACM SIGSOFT Symposium on the Foundations of Software Engineering (ESEC/FSE), 416–419.
- Just, R., Jalali, D., & Ernst, M. D. (2014). Defects4J: A database of existing faults to enable controlled testing studies for Java programs. Proceedings of the International Symposium on Software Testing and Analysis (ISSTA), 437–440.
- Panichella, A., Kifetew, F. M., & Tonella, P. (2018). Automated test case generation as a many-objective optimisation problem with dynamic selection of the targets. IEEE Transactions on Software Engineering, 44(2), 122–158.
- Schäfer, M., Nadi, S., Eghbali, A., & Tip, F. (2024). An empirical evaluation of using large language models for automated unit test generation. IEEE Transactions on Software Engineering, 50(1), 85–105.
- Shamshiri, S., Just, R., Rojas, J. M., Fraser, G., McMinn, P., & Arcuri, A. (2015). Do automatically generated unit tests find real faults? An empirical study of effectiveness and challenges. Proceedings of the 30th IEEE/ACM International Conference on Automated Software Engineering (ASE), 201–211.
- EclEmma. (n.d.). JaCoCo Java Code Coverage Library. <https://www.jacoco.org/jacoco/>
- Link, J. (n.d.). jqwik User Guide. <https://jqwik.net/docs/current/user-guide.html>
- ProjectSQA. (2569). Source code, test code and experimental results. <https://github.com/kaokao31/ProjectSQA>

ภาคผนวก ก

Prompt Template

Prompt Template ที่ใช้กับ Gemini และ DeepSeek (ไฟล์ Gemini/Prompt/master_prompt.md) โดย <TargetClassName> ถูกแทนด้วยชื่อคลาสเป้าหมายของแต่ละ bug

You are a Principal Software Quality Assurance Engineer specializing in Java Unit Testing with JUnit 4 and the Defects4J benchmark suite.

[OBJECTIVE]

Generate a complete, production-ready, executable JUnit 4 Test Suite class named ``<TargetClassName>Test.java`` for the provided Java source file.

The test suite must be designed to:

1. Achieve maximum Code Coverage (Line Coverage and Branch Coverage).
2. Trigger and reveal potential underlying faults/bugs (Fault Detection) present in the source code.

[STRICT OUTPUT CONSTRAINTS]

- Output ONLY pure executable Java code inside a single ````java ... ```` code block.
- DO NOT output any introductory text, Markdown explanations, analysis, summaries, or concluding remarks before or after the code block.
- The output MUST start directly with the package declaration (or imports) and end with the closing brace of the Java class.

[TECHNICAL & COMPATIBILITY REQUIREMENTS]

1. Package Matching: Read the ``package`` statement from the provided Java source file and use the EXACT SAME package name at the top of the generated test file.
2. Framework & Version: Must be strictly compatible with JDK 8 and JUnit 4 (``org.junit.Test``, ``org.junit.Assert.*``, ``org.junit.Before``, etc.). Do NOT use JUnit 5 (Jupiter) or features from JDK 9+.
3. Naming Convention: The test class name must be ``<TargetClassName>Test``.

[TEST GENERATION STRATEGY]

Use internal Chain-of-Thought reasoning to:

- Identify all decision branches (if/else, switch, loops, try/catch).
- Target edge cases, null checks, empty bounds, boundary values, and arithmetic limits.
- Craft specific input triggers designed to execute buggy paths inherent in Defects4J datasets.
- Ensure all assertions use standard JUnit 4 methods (`assertEquals`, `assertTrue`, `assertFalse`, `assertNull`, `assertNotNull`, `assertSame`).

สคริปต์เพิ่มบล็อกบริบทต่อท้ายสำหรับ 737 bugs ต่อโมเดล โดย 55 bugs ของ Lang มี Triggering Test และ Failure Reason จริง อีก 682 bugs มีค่า N/A ส่วน 117 bugs ไม่มีบล็อกนี้ รูปแบบบล็อกบริบทเป็นดังนี้

[TARGET CLASS & BUG CONTEXT]

- Project: <project>
- Bug ID: <bug id>
- Target Class: <fully qualified class name>
- Package: <package>
- Triggering Failing Test: <test name>
- Failure Reason: <failure cause>

ภาคผนวก ข

Reproducibility Guide

ขั้นตอนการทำซ้ำการทดลอง (รันจากโฟลเดอร์ของ Repository)

```
git clone https://github.com/kaokao31/ProjectSQA.git && cd ProjectSQA
```

```
docker compose -f docker/docker-compose.yml up -d --build
```

```
docker compose -f docker/docker-compose.yml exec sqa bash
```

```
bash scripts/check_env.sh
```

```
# EvoSuite (Search Budget 60 และ 120 วินาที)
```

```
python3 scripts/run_benchmark.py
```

```
# jqwik: controls -> Round 1 (200 tries) -> Round 2 (1000 tries) -> summary
```

```
JOBS=4 bash scripts/run_jqwik_auto_all.sh
```

```
# Gemini / DeepSeek: สร้าง Test Code แล้วประเมิน
```

```
cp .env.example .env
```

```
# แก้ .env: ใส่ API key, base URL และชื่อโมเดล (ดู README หัวข้อ 5.4)
```

```
python3 scripts/run_llm_testgen.py --provider all --all-projects
```

```
python3 scripts/run_llm_eval.py --providers Gemini,deepseek --jobs 4
```

```
# ตารางเปรียบเทียบทุกเทคนิค
```

```
python3 scripts/summarize_all.py
```

หมายเหตุ: ควรจัดสรรหน่วยความจำให้ Docker ตามทรัพยากรเครื่องและเริ่มต้นด้วยจำนวนงานพร้อมกันต่ำก่อน จากนั้นเพิ่มจำนวนงานเมื่อยืนยันว่าหน่วยความจำเพียงพอ การทดลองนี้รันพร้อมกัน 4 งานบนเครื่อง RAM 16 GB การรันพร้อมกันมากกว่านี้อาจเกินทรัพยากรของเครื่อง หากแต่ละงานใช้ประมาณ 2.5 GB

ภาคผนวก ค

โครงสร้างผลลัพธ์

ตารางที่ ค.1 โครงสร้างไฟล์และโฟลเดอร์หลักใน Repository

ตำแหน่ง	เนื้อหา
docker/	Dockerfile และ docker-compose.yml
scripts/	สคริปต์สร้าง Test, ประเมินผล และสรุปผล
dataset/defects4j/	รายการ bugs และ Target Classes ของแต่ละ project
EvoSuite/	Test Suite ที่ EvoSuite สร้าง และผลรายรอบ
jqwik/Harness/	ตัวสร้าง property อัตโนมัติและ runtime
jqwik/Auto/	property ที่สร้างขึ้นของแต่ละ bug
jqwik/Result_auto_Round1, Round2/	ผลราย bug ราย seed ของ jqwik
jqwik/Controls/	ชุดควบคุมสำหรับตรวจสอบ Pipeline
Gemini/, deepseek/	Prompt, TestCode และ Result ของแต่ละโมเดล
results/evosuite_results.csv	ผลของ EvoSuite
results/jqwik_auto.csv	ผลดิบของ jqwik
results/jqwik_auto_summary_*.csv	ผลสรุปของ jqwik ราย bug และราย project
results/llm_results.csv	ผลการประเมิน Gemini และ DeepSeek
results/comparison_*.csv	ตารางเปรียบเทียบทุกเทคนิค